ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

Đồ án Hệ thống Nhúng Thời gian thực

Internet vạn vật

LỚP: TN01

Giảng viên: Lê Trọng Nhân

STT	Họ và tên	MSSV	Tên lớp	Tên ngành
1	Nguyễn Lưu Khánh Trình	2313638	TN01	Kỹ thuật máy tính
2	Trần Minh Trí	2313626	TN01	Kỹ thuật máy tính

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 4 NĂM 2026

Mục lục

Tóm tắt **3**
Lời cảm ơn **4**
1 Giới thiệu **7**
1.1 Tổng quan đề tài . . . 7
1.2 Các nhiệm vụ và yêu cầu của đồ án . . . 7
1.3 Phạm vi và giới hạn . . . 7
1.4 Bố cục báo cáo . . . 8
2 Khái niệm hệ thống và cơ sở lý thuyết **9**
2.1 Giới thiệu . . . 9
2.2 Nền tảng IoT . . . 9
2.2.1 Thông số kỹ thuật chính . . . 9
2.2.2 Lý do lựa chọn ESP32 . . . 9
2.2.3 Các tính năng liên quan đến đồ án . . . 10
2.2.4 So sánh với các nền tảng thay thế . . . 10
2.3 Cơ sở về RTOS . . . 10
2.3.1 Khái niệm cốt lõi . . . 10
2.3.2 Ưu điểm của FreeRTOS . . . 11
2.3.3 Vai trò của FreeRTOS trong đồ án . . . 11
2.3.4 Kiến trúc tác vụ sử dụng trong đồ án . . . 12
2.3.5 So sánh với thiết kế firmware không dùng RTOS . . . 12
2.4 Cơ sở về TinyML . . . 12
2.4.1 Khái niệm cốt lõi . . . 12
2.4.2 Ưu điểm của TinyML . . . 13
2.4.3 Quy trình phát triển TinyML . . . 13
2.4.4 Ứng dụng TinyML trong đồ án . . . 13
2.4.5 Kiến trúc suy luận trong đồ án . . . 14
2.4.6 TinyML trên thiết bị so với ML trên đám mây . . . 14
2.4.7 Công cụ và thư viện hỗ trợ . . . 14
2.4.8 Các thách thức thực tế . . . 15
2.5 Cơ sở về CoreIoT . . . 15
2.6 Cơ sở về CoreIoT . . . 15
2.6.1 Luồng làm việc trên đám mây và Đường truyền dữ liệu . . . 15
2.6.2 Các thành phần giám sát . . . 15
2.7 Tóm tắt chương . . . 15
3 Thiết kế và triển khai hệ thống **16**
3.1 Giới thiệu . . . 16
3.2 Nhiệm vụ 1: Nhấp nháy LED theo nhiệt độ . . . 16
3.2.1 Nguyên lý hoạt động . . . 16
3.2.2 Cơ chế đồng bộ bằng FreeRTOS Semaphore . . . 16
3.2.3 Sơ đồ máy trạng thái (FSM) . . . 17
3.3 Nhiệm vụ 2: Điều khiển NeoPixel theo độ ẩm . . . 17
3.3.1 Nguyên lý hoạt động . . . 17
3.3.2 Cơ chế đồng bộ bằng FreeRTOS Semaphore . . . 18
3.3.3 Sơ đồ máy trạng thái (FSM) . . . 19
3.4 Nhiệm vụ 3: Giám sát nhiệt độ và độ ẩm với LCD . . . 19
3.4.1 Nguyên lý hoạt động . . . 19

3.4.2 Cơ chế điều phối hệ thống bằng Semaphore 20
3.4.3 Sơ đồ máy trạng thái (FSM) 21
3.5 Nhiệm vụ 4: Máy chủ web ở chế độ Access Point 22
3.5.1 Kiến trúc tổng thể 22
3.5.2 Quản lý mạng Wi-Fi và NVS (Non-Volatile Storage) 22
3.5.3 Backend Web Server (API và Routing) 23
3.5.4 Giao diện Frontend (HTML, CSS và JavaScript) 23
3.5.5 Quản lý tác vụ và Luồng xử lý (Task Management) 23
3.6 Nhiệm vụ 5: Triển khai TinyML và đánh giá độ chính xác 25
3.6.1 Thu thập dữ liệu từ cảm biến (Data Collection) 25
3.6.2 Tiền xử lý và Gán nhãn dữ liệu (Data Preprocessing and Labeling) . 25
3.7 Đánh giá và So sánh Mô hình (Model Benchmarking) 26
3.7.1 Huấn luyện và Xuất mã C (Model Training and C Header Export) . 27
3.7.2 Kiểm thử mô hình cục bộ (Local Model Testing) 28
3.7.3 Triển khai Firmware trên Vi điều khiển (On-device Deployment) . . 29
3.8 Nhiệm vụ 6: Gửi dữ liệu lên máy chủ đám mây CoreIoT 32
3.8.1 Nguyên lý hoạt động 32
3.8.2 Sơ đồ máy trạng thái (FSM) 33
3.9 Tổng quan kiến trúc 33
3.10 Tính mới và cải tiến 33
3.11 Tóm tắt chương 33
4 Đánh giá và kết quả thực nghiệm 34
4.1 Giới thiệu 34
4.2 Các kịch bản kiểm thử 34
4.3 Kết quả chức năng 34
4.4 Đánh giá TinyML 34
4.5 Thảo luận 34
4.6 Tóm tắt chương 34
5 Kết luận và thảo luận nhóm 35
5.1 Thảo luận và kết luận 35
5.2 Hướng phát triển trong tương lai 35
Tài liệu tham khảo 36
A Phụ lục A: Tổ chức nhóm 37
B Phụ lục B: Tài liệu bổ sung 38

Tóm tắt

[Viết một phần tóm tắt ngắn gọn: mục tiêu, phương pháp, kết quả chính và từ khóa.]

Lời cảm ơn

[Viết lời cảm ơn dành cho giảng viên, các thành viên nhóm và những tài liệu tham khảo hữu ích.]

Danh sách hình

1 Task 1 FSM 17
2 Task 2 FSM 19
3 Task 3 FSM 21
4 Task 6 FSM 33

Danh sách bảng

1 Giới thiệu

1.1 Tổng quan đề tài

Đồ án này xây dựng một hệ thống IoT thông minh dựa trên nền tảng ESP32 để giám sát và điều khiển môi trường theo thời gian thực. Hệ thống thu thập dữ liệu nhiệt độ và độ ẩm từ các cảm biến tích hợp, hiển thị kết quả qua giao diện web, đồng thời cho phép người dùng cấu hình Wi-Fi, theo dõi trạng thái thiết bị và điều khiển từ xa một số chức năng đã chọn. Bên cạnh đó, đồ án tích hợp mô hình Machine Learning/TinyML để phân tích dữ liệu cảm biến và đưa ra dự đoán hoặc khuyến nghị phù hợp với trạng thái môi trường quan sát được. Giải pháp đề xuất hướng đến tự động hóa, cải thiện trải nghiệm người dùng và vẫn phù hợp với giới hạn tài nguyên xử lý và bộ nhớ của vi điều khiển ESP32.

1.2 Các nhiệm vụ và yêu cầu của đồ án

Phần triển khai được tổ chức thành sáu nhiệm vụ chính:

1. **Nhiệm vụ 1: Nhấp nháy LED theo nhiệt độ** — điều khiển LED trạng thái nhấp nháy dựa trên các mức nhiệt độ đo được.
2. **Nhiệm vụ 2: Điều khiển NeoPixel theo độ ẩm** — thay đổi màu sắc/hành vi của NeoPixel dựa trên các khoảng độ ẩm.
3. **Nhiệm vụ 3: Giám sát nhiệt độ và độ ẩm với LCD** — đọc dữ liệu cảm biến liên tục và hiển thị kết quả thời gian thực trên giao diện LCD.
4. **Nhiệm vụ 4: Máy chủ web ở chế độ Access Point** — triển khai máy chủ web trên ESP32-S3 để cung cấp chức năng cấu hình và giám sát cục bộ qua chế độ AP.
5. **Nhiệm vụ 5: Triển khai TinyML và đánh giá độ chính xác** — đưa mô hình TinyML lên thiết bị và đánh giá chất lượng suy luận bằng các thước đo phù hợp.
6. **Nhiệm vụ 6: Gửi dữ liệu lên máy chủ đám mây CoreIoT** — truyền dữ liệu thu thập được và trạng thái thiết bị đến backend CoreIoT để giám sát từ xa.

Toàn bộ hoạt động lập trình, tích hợp và kiểm thử được thực hiện trên nền tảng **ESP32-S3** với **PlatformIO** làm môi trường phát triển chính.

1.3 Phạm vi và giới hạn

[Mô tả những nội dung mà báo cáo bao quát và những phần nằm ngoài phạm vi.]

1.4 Bố cục báo cáo

[Trình bày ngắn gọn cấu trúc các chương còn lại của báo cáo.]

2 Khái niệm hệ thống và cơ sở lý thuyết

2.1 Giới thiệu

[Giới thiệu khái niệm hệ thống và lý do cần trình bày phần cơ sở lý thuyết.]

2.2 Nền tảng IoT

ESP32 là họ vi điều khiển 32-bit do Espressif Systems phát triển, kế thừa từ ESP8266. Nền tảng này tích hợp kết nối Wi-Fi và Bluetooth/BLE, đồng thời cung cấp hiệu năng xử lý đủ tốt cho các ứng dụng IoT biên. Trong đồ án này, ESP32-S3 được dùng làm bộ điều khiển trung tâm cho các tác vụ cảm biến, xử lý cục bộ, truyền thông và tương tác với đám mây.

2.2.1 Thông số kỹ thuật chính

- **Processor:** Xtensa/RISC-V class 32-bit architecture (model-dependent), typically operated in the 160–240 MHz range.
- **Memory:** On-chip SRAM resources with external Flash support (commonly 4–16 MB, depending on board configuration).
- **Connectivity:** IEEE 802.11 b/g/n Wi-Fi and Bluetooth Low Energy.
- **Peripheral Interfaces:** GPIO, PWM, ADC, UART, SPI, and I^2C.
- **Power Features:** Support for low-power modes suitable for embedded and battery-aware IoT systems.
- **Operating Range:** Industrial-grade operating conditions for practical deployment scenarios.

2.2.2 Lý do lựa chọn ESP32

Nền tảng ESP32 được lựa chọn vì các lý do sau:

1. **Integrated wireless connectivity:** native Wi-Fi support simplifies device-to-cloud communication and local web control.
2. **Balanced performance:** sufficient processing capability to execute sensor pipelines, web services, and lightweight TinyML inference on one device.
3. **Practical memory capacity:** enough resources for buffering sensor data, serving web assets, and running compact inference models.
4. **Strong software ecosystem:** mature support through PlatformIO, Arduino-compatible libraries, and broad community documentation.

5. **Cost-effectiveness:** favorable hardware cost compared to other IoT platforms with similar connectivity features.
6. **Energy efficiency:** suitable for long-running embedded applications and power-constrained scenarios.

2.2.3 Các tính năng liên quan đến đồ án

Trong đồ án này, các khả năng sau của ESP32 được sử dụng trực tiếp:

- **I^{2}C interface:** communication with digital sensors for temperature and humidity acquisition.
- **GPIO/PWM:** control of status LEDs, NeoPixel behavior, and other actuators.
- **Web server support:** local monitoring and control in Access Point mode.
- **Flash file system support (e.g., LittleFS):** storage of static web resources (HTML/CSS/JS).
- **TinyML compatibility:** on-device inference for lightweight environmental state analysis.

2.2.4 So sánh với các nền tảng thay thế

So với các lựa chọn phổ biến như Arduino Uno, Arduino MKR WiFi hoặc các board thuộc họ Raspberry Pi Pico, ESP32 cân bằng tốt hơn giữa năng lực tính toán, kết nối không dây tích hợp, hỗ trợ phần mềm và chi phí triển khai. Nhờ đó, ESP32 đặc biệt phù hợp với một đồ án IoT toàn diện, kết hợp cảm biến, trí tuệ cục bộ, tương tác web và gửi dữ liệu lên đám mây.

2.3 Cơ sở về RTOS

FreeRTOS là một nhân hệ điều hành thời gian thực (RTOS) gọn nhẹ, mã nguồn mở, được thiết kế cho các hệ thống nhúng có tài nguyên hạn chế. Hệ điều hành này cung cấp một lớp trừu tượng có cấu trúc cho thực thi đồng thời, xử lý sự kiện và đồng bộ giữa các tác vụ mà không yêu cầu lập trình viên phải tự xây dựng cơ chế lập lịch phức tạp. Nhờ chi phí tài nguyên thấp và hệ sinh thái hỗ trợ trưởng thành, FreeRTOS được sử dụng rộng rãi trên các nền tảng vi điều khiển như ESP32, STM32 và ARM Cortex-M.

2.3.1 Khái niệm cốt lõi

- **Task:** an independent execution unit, typically implemented as an infinite loop with its own stack and runtime state.
- **Scheduler:** the kernel component that decides which task executes at each instant; FreeRTOS commonly uses preemptive priority-based scheduling.

- **Queue:** a thread-safe communication mechanism for passing messages or data between tasks.
- **Semaphore:** a synchronization primitive used for signaling events and controlling access to shared resources.
- **Mutex:** a specialized locking mechanism for protecting critical shared resources from concurrent access.

2.3.2 Ưu điểm của FreeRTOS

FreeRTOS mang lại nhiều lợi ích thực tiễn cho đồ án này:

1. **Multitasking support:** multiple functional components can run concurrently as independent tasks.
2. **Open-source flexibility:** the kernel is freely available and can be adapted to project-specific constraints.
3. **Low resource footprint:** suitable for microcontrollers with limited memory and compute resources.
4. **Modularity and scalability:** queue, semaphore, software timer, and synchronization features can be incrementally integrated.
5. **Strong platform support:** deeply integrated with ESP-IDF and well supported by the embedded systems community.
6. **Industrial reliability:** broadly validated in large-scale commercial embedded deployments.

2.3.3 Vai trò của FreeRTOS trong đồ án

Trong hệ thống IoT này, FreeRTOS được dùng để điều phối nhiều mô-đun có yêu cầu thời gian và truyền thông cao:

1. **Concurrent task management:** separate tasks handle web serving, periodic sensor sampling, TinyML inference, and LED/device control.
2. **Data synchronization:** semaphores protect shared sensor data paths, while queues support safe task-to-task data transfer.
3. **Priority-aware response:** web request handling can be assigned higher priority for responsive user interaction; sensing and inference tasks run at controlled medium priorities.
4. **Resource isolation:** each task has dedicated stack allocation, reducing memory interference and improving system stability.

2.3.4 Kiến trúc tác vụ sử dụng trong đồ án

Thiết kế phần mềm tuân theo mô hình tổ chức tác vụ nhiều lớp của FreeRTOS:

- **Task WebServer (high priority):** handles HTTP requests and local UI interactions.
- **Task Sensor Reader (medium priority):** acquires periodic temperature and humidity measurements.
- **Task TinyML (medium priority):** executes on-device inference for state prediction.
- **Task LED/Device Control:** updates LED/NeoPixel and related actuator outputs.
- **Task Wi-Fi Manager:** manages connectivity and reconnection logic.
- **Idle Task (lowest priority):** runs when no application task is ready.

2.3.5 So sánh với thiết kế firmware không dùng RTOS

Nếu không có RTOS, việc phát triển firmware thường phải dựa nhiều vào cơ chế ngắt và các máy trạng thái được quản lý thủ công, khiến hệ thống khó mở rộng và bảo trì khi độ phức tạp tăng lên. FreeRTOS khắc phục các hạn chế này bằng cách phân rã ứng dụng thành các tác vụ độc lập và giao việc lập lịch, đồng bộ cho nhân hệ điều hành, từ đó cải thiện khả năng bảo trì, tính dự đoán về thời gian và độ ổn định trước lỗi tranh chấp tài nguyên.

2.4 Cơ sở về TinyML

TinyML (Tiny Machine Learning) tập trung vào việc triển khai suy luận học máy trên các thiết bị nhúng có tài nguyên hạn chế như vi điều khiển và cảm biến thông minh. Thay vì chuyển luồng dữ liệu cảm biến thô lên máy chủ đám mây để xử lý AI, TinyML thực hiện dự đoán trực tiếp ở biên. Cách tiếp cận này giúp giảm độ trễ, giảm chi phí năng lượng cho truyền thông, tăng cường riêng tư và cho phép hệ thống tiếp tục hoạt động ngay cả khi mạng không ổn định.

2.4.1 Khái niệm cốt lõi

- **Inference:** using a pre-trained model to predict new input samples; TinyML primarily targets inference rather than on-device training.
- **Model quantization:** reducing model precision (e.g., from float32 to int8/int16) to decrease memory usage and improve runtime efficiency.
- **TensorFlow Lite / TensorFlow Lite Micro:** lightweight runtimes for deploying compact models on embedded and IoT hardware.
- **Feature extraction:** converting raw sensor readings into meaningful features suitable for model input.

- **Edge AI:** running AI computation at the data source rather than in remote cloud infrastructure.

2.4.2 Ưu điểm của TinyML

Trong đồ án này, TinyML mang lại một số lợi ích thực tiễn:

1. **Low latency:** predictions are produced locally in near real time.
2. **Energy efficiency:** reduced network transmission lowers power consumption.
3. **Data privacy:** sensitive raw measurements can remain on-device.
4. **Offline capability:** core inference continues even when cloud connectivity is unavailable.
5. **Compact deployment:** optimized models can run within limited memory and compute budgets.
6. **Infrastructure cost reduction:** less dependency on high-capacity cloud inference services.

2.4.3 Quy trình phát triển TinyML

Quy trình triển khai thường bao gồm các bước sau:

1. Train a model offline using Python-based ML frameworks and historical datasets.
2. Convert the trained model to a lightweight deployment format (e.g., `.tflite` or equivalent compact representation).
3. Apply optimization/quantization to reduce memory footprint and improve execution speed.
4. Embed model parameters into firmware (for example as a C header file).
5. Deploy to ESP32-S3 and evaluate runtime latency, memory usage, and prediction quality.

2.4.4 Ứng dụng TinyML trong đồ án

Trong hệ thống IoT này, TinyML được sử dụng để phân loại trạng thái môi trường từ dữ liệu cảm biến thời gian thực:

- **Input features:** temperature (°C) and humidity (%) from the DHT20 sensor.
- **Output classes:** environmental operating states (e.g., invalid state, standby, cooling required, dehumidification required, humidification required).
- **Model choice:** Gaussian Naive Bayes (GNB), trained on historical labeled pairs of temperature-humidity samples.

- **Firmware integration:** model parameters exported to `naive_bayes_model.h` and compiled into the ESP32-S3 application.

To reduce communication overhead, the system can publish compact inference results (state labels) instead of transmitting all raw samples continuously.

2.4.5 Kiến trúc suy luận trong đồ án

Luồng suy luận được tổ chức như sau:

1. Acquire temperature and humidity from DHT20.
2. Apply preprocessing/feature formatting.
3. Run Gaussian Naive Bayes inference on ESP32-S3.
4. Produce the environmental-state classification.
5. Publish results to local UI and cloud dashboard.

2.4.6 TinyML trên thiết bị so với ML trên đám mây

Criterion	On-Device TinyML	Cloud ML
Latency	Very low (typically milliseconds)	Higher due to communication and server round-trip
Energy usage	Lower communication overhead	Higher network usage for continuous data upload
Data privacy	Raw data can remain on-device	Raw data is usually transmitted to remote servers
Network independence	Can continue inference offline	Requires stable connectivity for inference
Scalability	Limited by device resources	Scales with cloud infrastructure capacity
Model update flow	Often requires firmware/model redeployment	Centralized updates are easier on server side

2.4.7 Công cụ và thư viện hỗ trợ

Các công cụ phổ biến cho quy trình làm việc theo hướng TinyML gồm TensorFlow Lite, TensorFlow Lite Micro, các pipeline huấn luyện dựa trên scikit-learn và bộ SDK/thư viện nhúng sẵn có trong hệ sinh thái ESP32.

2.4.8 Các thách thức thực tế

Những ràng buộc chính của TinyML khi triển khai trên thiết bị nhúng bao gồm giới hạn chặt chẽ về RAM/Flash, khả năng suy giảm độ chính xác sau khi lượng tử hóa mạnh, và việc gỡ lỗi trên phần cứng khó hơn nhiều so với môi trường máy tính để bàn hoặc đám mây.

2.5 Cơ sở về CoreIoT

[Trình bày ngắn gọn luồng làm việc trên đám mây CoreIoT, đường truyền dữ liệu và các thành phần giám sát.]

2.6 Cơ sở về CoreIoT

2.6.1 Luồng làm việc trên đám mây và Đường truyền dữ liệu

CoreIoT là nền tảng quản lý IoT thời gian thực. Hệ thống sử dụng MQTT - giao thức truyền thông Publish/Subscribe hạng nhẹ, đặc biệt tối ưu cho vi điều khiển ESP32. Luồng dữ liệu hai chiều được định tuyến như sau:

- **Chiều lên (Telemetry):** Trạng thái môi trường và thông số ngưỡng được đóng gói thành định dạng JSON, xuất bản (Publish) định kỳ lên đám mây tại topic `v1/devices/me/telemetry`.
- **Chiều xuống (RPC Control):** Đám mây đóng vai trò Broker điều hướng lệnh điều khiển. ESP32 đăng ký lắng nghe (Subscribe) tại topic `v1/devices/me/rpc/request/+` để bắt sự kiện từ người dùng và phản hồi xuống phần cứng.

2.6.2 Các thành phần giám sát

Không gian điều khiển của hệ thống được tích hợp trên Dashboard thông qua hai nhóm thành phần chính:

- **Thành phần hiển thị:** Gồm Thẻ số (Digital Cards) và Biểu đồ đường (Line Charts) để giám sát và lưu trữ chuỗi dữ liệu (time-series) của cảm biến.
- **Thành phần điều khiển:** Gồm thanh trượt (Slider) và công tắc (Switch) liên kết với các phương thức RPC (`setThre`, `setStateLED`). Điều này cho phép thay đổi logic điều khiển trực tiếp trên nền tảng Web.

2.7 Tóm tắt chương

Chương này đã trình bày quá trình thiết kế và triển khai phần mềm cho hệ thống IoT. Nhờ ứng dụng hệ điều hành FreeRTOS, ESP32 đã vận hành trên kiến trúc đa nhiệm gồm: module thu thập dữ liệu (DHT20, LCD), cảnh báo trực quan (LED, NeoPixel), Web Server thiết lập mạng và giao tiếp đám mây CoreIoT qua MQTT.

3 Thiết kế và triển khai hệ thống

3.1 Giới thiệu

[Giới thiệu chương triển khai và quy trình thiết kế.]

3.2 Nhiệm vụ 1: Nhấp nháy LED theo nhiệt độ

[Mô tả logic điều khiển, điều kiện kích hoạt và cách sử dụng semaphore.]

3.2.1 Nguyên lý hoạt động

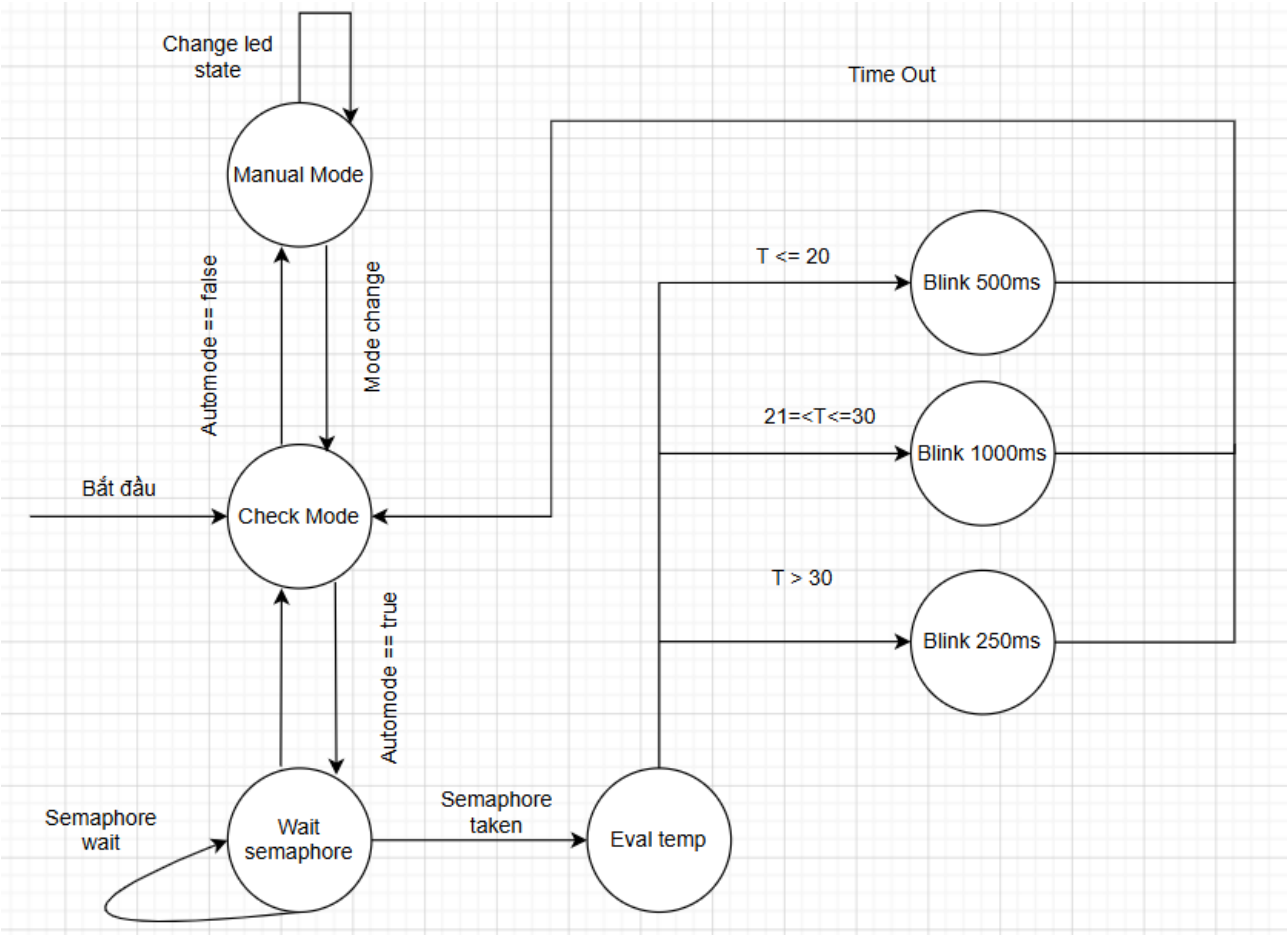
Tác vụ "Temperature-Driven LED Blinking"được thiết kế nhằm điều khiển trạng thái chớp tắt của đèn cảnh báo LED1 thông qua dữ liệu nhiệt độ môi trường hoặc sự can thiệp trực tiếp từ người dùng. Hệ thống hoạt động dựa trên cơ chế chuyển đổi linh hoạt giữa hai trạng thái chính: *Auto Mode* (Chế độ tự động) và *Manual Mode* (Chế độ thủ công).

- **Manual Mode (Chế độ thủ công):** Khi cờ `led_auto_mode` được đặt thành `false`, hệ thống sẽ bỏ qua các dữ liệu từ cảm biến. Trạng thái của LED1 lúc này được quyết định hoàn toàn bởi biến `led_manual_1`, cho phép phản hồi tức thời với các tín hiệu điều khiển từ xa qua Web Server hoặc nền tảng CoreIoT.
- **Auto Mode (Chế độ tự động):** Khi `led_auto_mode` là `true`, tác vụ sẽ chờ tín hiệu đồng bộ (Semaphore) từ quá trình đọc cảm biến. Tần số chớp tắt của LED được chia thành 3 dải nhiệt (T) như sau:
  - $T \leq 20°C$: Môi trường lạnh, chu kỳ nhấp nháy là 500ms.
  - $21°C \leq T \leq 30°C$: Môi trường ổn định, chu kỳ nhấp nháy chậm ở mức 1000ms.
  - $T > 30°C$: Cảnh báo nhiệt độ cao, chu kỳ nhấp nháy nhanh ở mức 250ms để thu hút sự chú ý.

3.2.2 Cơ chế đồng bộ bằng FreeRTOS Semaphore

Được triển khai dưới dạng một FreeRTOS Task độc lập (`led_blinky`), hệ thống tránh được hiện tượng thắt cổ chai (blocking) khi xử lý các hàm `vTaskDelay`. Nhằm đảm bảo tính toàn vẹn dữ liệu (Thread-safe) giữa Task đọc cảm biến và Task điều khiển LED, một Binary Semaphore (`xBinarySemaphoreTemp_blinky`) đã được áp dụng. Task LED chỉ lấy giá trị nhiệt độ mới vào biến cục bộ `local_temp` khi chiếm được quyền kiểm soát Semaphore, qua đó ngăn chặn tuyệt đối lỗi "Data Race".

3.2.3 Sơ đồ máy trạng thái (FSM)



Hình 1: Task 1 FSM

3.3 Nhiệm vụ 2: Điều khiển NeoPixel theo độ ẩm

[Mô tả mẫu màu, các ngưỡng và cơ chế phối hợp dựa trên semaphore.]

3.3.1 Nguyên lý hoạt động

Tác vụ "Humidity and Network-Driven NeoPixel Control"chịu trách nhiệm điều khiển đèn LED RGB (NeoPixel) để phản ánh trực quan tình trạng độ ẩm không khí và trạng thái kết nối mạng của hệ thống. Tương tự như LED1, dải NeoPixel cũng hỗ trợ hai chế độ hoạt động: *Auto Mode* và *Manual Mode*.

- **Manual Mode (Chế độ thủ công):** Khi `led_auto_mode` bị tắt, màu sắc của NeoPixel được xác định bởi các giá trị `led_manual_r`, `led_manual_g`, `led_manual_b` và được tinh chỉnh độ sáng thông qua biến `led_brightness` (tính theo phần trăm). Sau khi cập nhật màu sắc, vòng lặp sẽ tạm nghỉ 150ms.

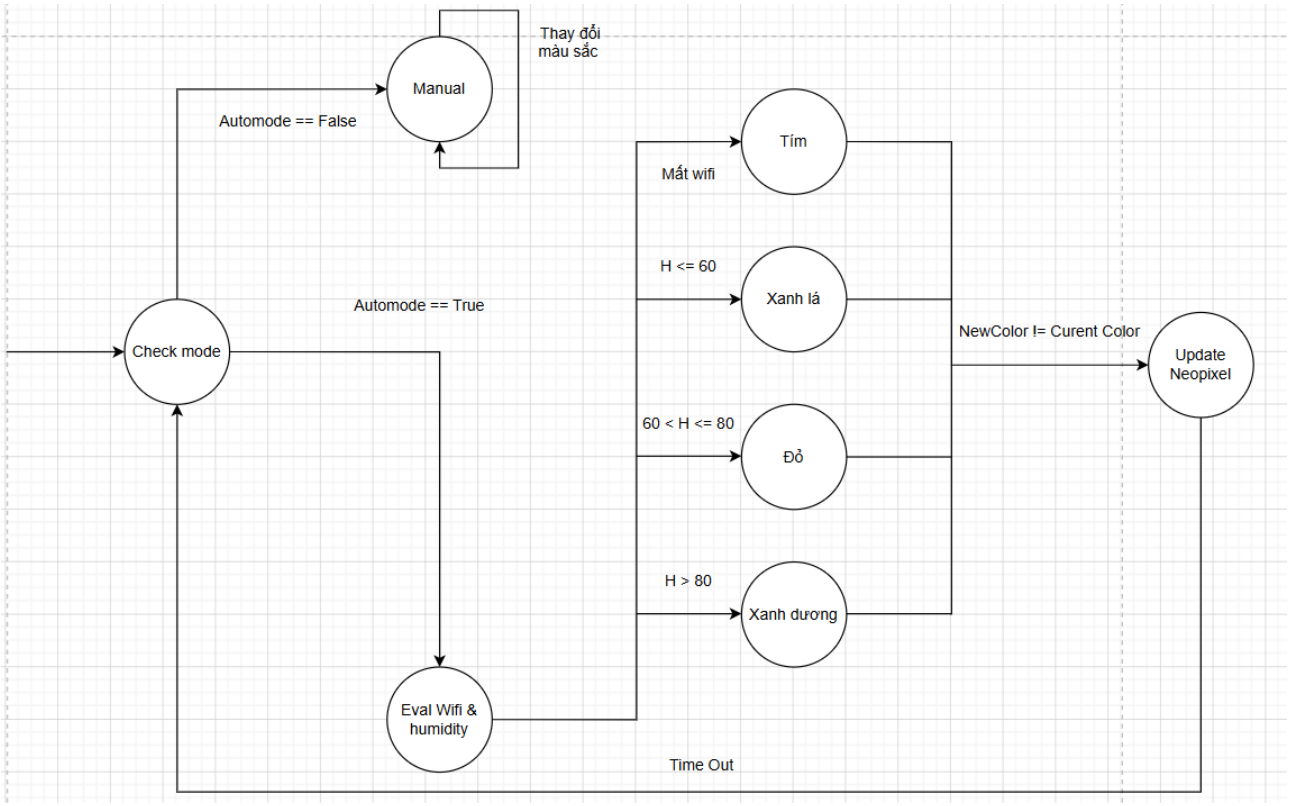
- **Auto Mode (Chế độ tự động):** Ở chế độ này, hệ thống sẽ ưu tiên kiểm tra trạng thái kết nối mạng, sau đó mới đánh giá dữ liệu độ ẩm (H) từ cảm biến. Các mức hiển thị màu sắc được quy định cụ thể như sau:
  - **Mất kết nối Wi-Fi** (`!isWifiConnected`): Đèn hiển thị màu Tím (Purple) cảnh báo mất mạng độc lập với dữ liệu cảm biến.
  - $H \leq 60\%$: Độ ẩm lý tưởng, đèn hiển thị màu Xanh lá (Green).
  - $60\% < H \leq 80\%$: Độ ẩm hơi cao, đèn hiển thị màu Đỏ (Red).
  - $H > 80\%$: Độ ẩm bão hòa, đèn hiển thị màu Xanh dương (Blue).

  Tác vụ chạy với chu kỳ delay là 300ms để đảm bảo cập nhật trạng thái liên tục mà không tiêu tốn tài nguyên hệ thống.

3.3.2 Cơ chế đồng bộ bằng FreeRTOS Semaphore

Tác vụ `neo_blinky` sử dụng hàm `xSemaphoreTake` để kiểm tra việc có hay không dữ liệu mới từ cảm biến. Khi nhận được Semaphore từ Task đọc cảm biến, biến cục bộ `local_hum` sẽ được cập nhật bằng giá trị mới nhất từ biến toàn cục `glob_humidity`. Điều này giúp vòng lặp của NeoPixel không bị chặn lại nếu cảm biến chưa đọc xong, duy trì khả năng cảnh báo mất Wi-Fi tức thời. Hệ thống cũng áp dụng kỹ thuật so sánh cờ `currentColor` và `newColor` để chỉ gọi hàm `strip.show()` khi thực sự có sự thay đổi màu sắc,

3.3.3 Sơ đồ máy trạng thái (FSM)



Hình 2: Task 2 FSM

3.4 Nhiệm vụ 3: Giám sát nhiệt độ và độ ẩm với LCD

[Giải thích việc lấy mẫu cảm biến định kỳ, định dạng dữ liệu và logic cập nhật LCD theo thời gian thực.]

3.4.1 Nguyên lý hoạt động

Tác vụ "Temperature and Humidity Monitoring"đóng vai trò là khối thu thập dữ liệu trung tâm (Producer) của toàn bộ hệ thống. Tác vụ này có trách nhiệm giao tiếp với cảm biến DHT20 qua chuẩn I2C để trích xuất nhiệt độ và độ ẩm, hiển thị trực quan lên màn hình LCD 16x2, đồng thời cung cấp dữ liệu đầu vào cho các tác vụ phân tích và cảnh báo khác.

Nguyên lý hoạt động được chia thành các bước kiểm duyệt và xử lý chặt chẽ:

- **Kiểm duyệt dữ liệu (Data Validation):** Sau mỗi lần đọc cảm biến, hệ thống sử dụng hàm `isnan()` để kiểm tra tính hợp lệ của dữ liệu. Nếu xuất hiện lỗi đọc (NaN), hệ thống sẽ in cảnh báo ra Serial, tạm dừng 1000ms và lặp lại quá trình đọc mà không làm thay đổi trạng thái của các biến toàn cục. Điều này giúp bảo vệ hệ thống khỏi các giá trị nhiễu hoặc rác.

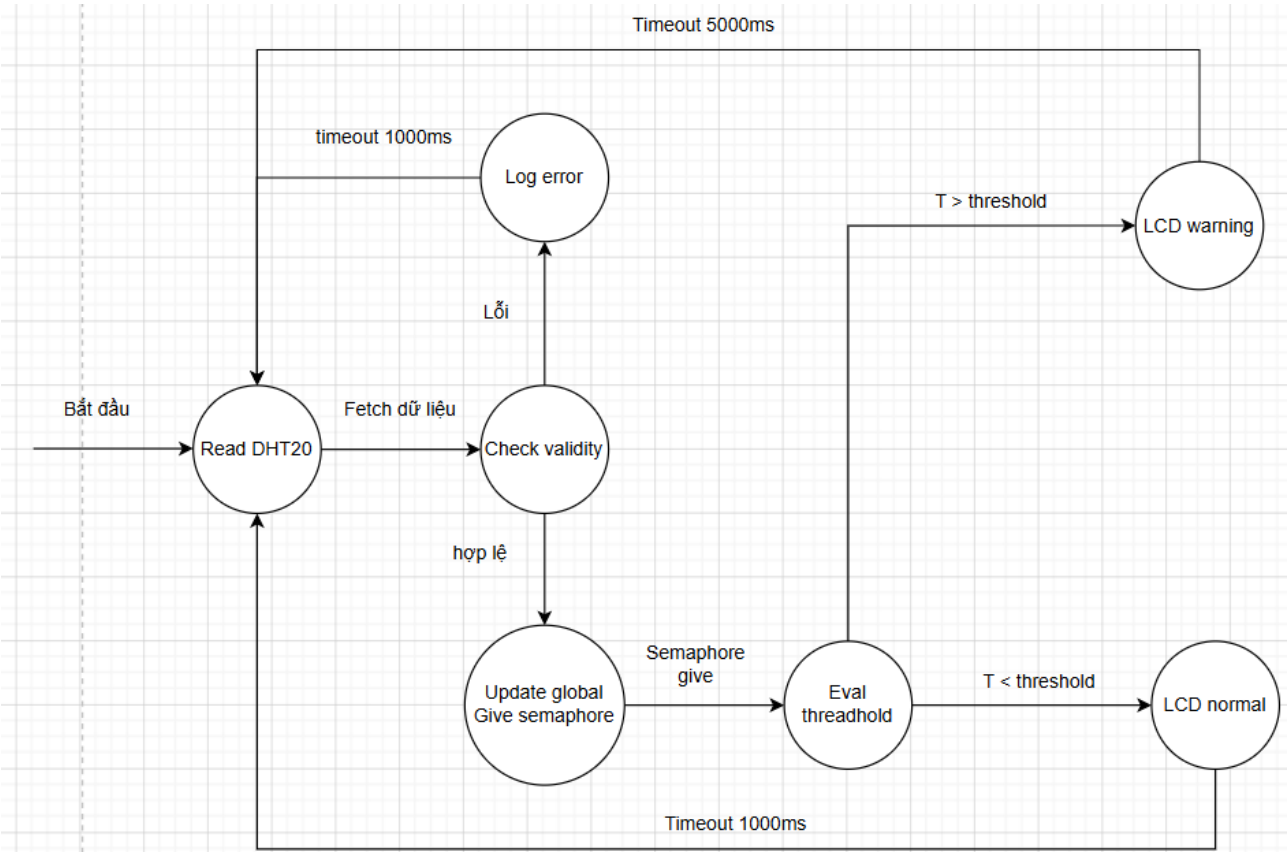
- **Hiển thị có điều kiện (Conditional Display):** Dữ liệu hợp lệ sẽ được so sánh với ngưỡng nhiệt độ `glob_temp_threshold` (có thể được cập nhật động từ nền tảng CoreIoT).
  - **Môi trường bình thường ($T \leq$ Threshold):** LCD hiển thị trực quan Độ ẩm (H) ở hàng trên và Nhiệt độ (T) ở hàng dưới. Hệ thống duy trì trạng thái này trong chu kỳ 1000ms trước khi cập nhật mẫu mới.
  - **Vượt ngưỡng cảnh báo ($T >$ Threshold):** LCD ngay lập tức chuyển sang chế độ báo động. Hàng dưới chốt cứng thông số nhiệt độ hiện tại kèm mức ngưỡng vi phạm. Hàng trên thực hiện hiệu ứng chớp tắt dòng chữ `"! WARNING TEMP !"`5 lần (chu kỳ 500ms sáng / 500ms tắt, tổng thời gian 5000ms) để thu hút tối đa sự chú ý của người dùng mà không làm gián đoạn hệ điều hành (nhờ `vTaskDelay`).

3.4.2 Cơ chế điều phối hệ thống bằng Semaphore

Khác với Task 1 và Task 2 đóng vai trò là "Consumer", Task giám sát nhiệt ẩm hoạt động như một "Producer". Ngay khi dữ liệu hợp lệ được ghi vào biến toàn cục `glob_temperature` và `glob_humidity`, tác vụ này lập tức phát tín hiệu đánh thức các phân hệ khác bằng cách giải phóng đồng thời 3 Binary Semaphore:

1. `xBinarySemaphoreTemp_blinky`: Kích hoạt tính toán lại tần số nháy LED1.
2. `xBinarySemaphoreTemp_neo`: Kích hoạt đánh giá màu sắc dải LED NeoPixel.
3. `xBinarySemaphoreTinyMLData`: Kích hoạt bộ nội suy Machine Learning (Naive Bayes) để dự đoán nhãn trạng thái môi trường.

3.4.3 Sơ đồ máy trạng thái (FSM)



Hình 3: Task 3 FSM

3.5 Nhiệm vụ 4: Máy chủ web ở chế độ Access Point

3.5.1 Kiến trúc tổng thể

Hệ thống được thiết kế theo mô hình **Client-Server đa luồng (Multi-tasking)** kết hợp với mô hình **Single Page Application (SPA)** ở phía giao diện:

- **Backend (ESP32):** Đóng vai trò là Web Server (cung cấp các API và giao diện HTML) và Router (quản lý kết nối mạng Wi-Fi ở cả chế độ AP và STA).
- **Frontend (Trình duyệt Web):** Giao diện được nhúng trực tiếp vào bộ nhớ Flash của vi điều khiển ESP32. Khi trình duyệt tải trang về, hệ thống sử dụng JavaScript (thông qua Fetch API) để liên tục gọi các API của Backend, từ đó cập nhật dữ liệu cảm biến và trạng thái mới nhất mà không cần tải lại trang (chống giật lag và tiết kiệm băng thông).
- **Hệ điều hành thời gian thực (FreeRTOS):** Toàn bộ hệ thống Web Server được đóng gói và chạy trên một luồng (Task) hoàn toàn riêng biệt có tên là `main_server_task`. Kiến trúc này cho phép các chức năng cốt lõi khác của hệ thống (như đọc cảm biến môi trường, xử lý tín hiệu, hay chạy mô hình AI TinyML) có thể hoạt động song song ở các Task khác mà không bị chặn (blocking) bởi các tiến trình mạng.

3.5.2 Quản lý mạng Wi-Fi và NVS (Non-Volatile Storage)

Hệ thống sử dụng linh hoạt hai chế độ mạng của vi điều khiển ESP32 bao gồm **AP (Access Point - trạm phát)** và **STA (Station - thiết bị kết nối)**:

- **AP Mode:** Chế độ này luôn được kích hoạt mặc định khi hệ thống khởi động (thông qua hàm `startAP()`). ESP32 tự đóng vai trò là một trạm phát Wi-Fi độc lập, cho phép thiết bị của người dùng kết nối trực tiếp vào để tiến hành cấu hình mạng ban đầu.
- **STA Mode:** Khi người dùng cung cấp thông tin mạng cục bộ (SSID và Mật khẩu) từ giao diện, thiết bị sẽ chuyển sang chế độ `WIFI_AP_STA` (hoạt động song song cả thu và phát). Nếu kết nối thành công tới router nội bộ, hệ thống sẽ ghi nhận và hiển thị địa chỉ IP được cấp phát.
- **Lưu trữ Preferences (NVS):** Để đảm bảo tính tự động hóa, hệ thống sử dụng thư viện `<Preferences.h>` nhằm lưu trữ vĩnh viễn dữ liệu cấu hình mạng vào phân vùng Flash (Non-Volatile Storage). Ở những lần khởi động tiếp theo, vi điều khiển sẽ tự động gọi hàm `loadWifiCredentials()` để trích xuất thông tin này và tái thiết lập kết nối mà không cần sự can thiệp của người dùng.

3.5.3 Backend Web Server (API và Routing)

Hệ thống sử dụng thư viện `<WebServer.h>` để định nghĩa các Endpoints (đường dẫn API). Cụ thể, trong hàm `setupServer()`, máy chủ web được cấu hình với 3 nhóm API cốt lõi:

1. **Phục vụ giao diện (HTML):** Các đường dẫn `/`, `/dashboard`, và `/settings`. Nhóm này có nhiệm vụ trả về toàn bộ chuỗi HTML (chứa CSS/JS) để hiển thị giao diện Single Page Application trên trình duyệt.
2. **API điều khiển & cấu hình:** Bao gồm `/toggle`, `/led-mode`, `/led-rgb`, `/connect`, và `/scan-networks`. Đây là các endpoint tiếp nhận lệnh từ người dùng để thay đổi trạng thái phần cứng (bật/tắt, đổi màu LED) hoặc tiến hành quét/kết nối mạng Wi-Fi.
3. **API cung cấp dữ liệu (JSON):** Bao gồm `/sensors`, `/history`, `/led-status`, và `/tinyml-status`. Giao diện web sẽ liên tục gọi các API này để cập nhật số liệu. Đặc biệt, thông qua hàm `handleTinyMlStatus()`, ta có thể thấy rõ cơ chế giao tiếp liên luồng (inter-task communication) giữa Web Task và AI Task thông qua việc đọc các biến toàn cục (ví dụ: `tinyml_predicted_label`).

3.5.4 Giao diện Frontend (HTML, CSS và JavaScript)

Thay vì sử dụng các tệp tin lưu trữ rời rạc, toàn bộ giao diện người dùng được nhúng trực tiếp vào mã nguồn dưới dạng các chuỗi ký tự định dạng `rawliteral`. Kiến trúc giao diện tập trung vào các đặc điểm sau:

- **Single Page Application (SPA):** Dashboard được thiết kế để hoạt động trên một trang duy nhất. Sử dụng thư viện `Chart.js` để trực quan hóa dữ liệu cảm biến. Giao diện có khả năng phản hồi (Responsive) tốt trên cả thiết bị di động và máy tính.
- **Cơ chế Fallback Canvas:** Đây là một điểm sáng trong thiết kế hệ thống. Trong điều kiện thiết bị không có kết nối Internet (không thể tải thư viện từ CDN), mã nguồn JavaScript sẽ tự động kích hoạt hàm `drawFallbackChart()` sử dụng `Canvas API` thuần để vẽ biểu đồ. Điều này đảm bảo tính sẵn sàng cao của hệ thống trong mọi môi trường mạng.
- **Cập nhật dữ liệu thời gian thực:** Sử dụng hàm `setInterval()` để tự động thực hiện các truy vấn `fetch()` tới các endpoint JSON (`/history`, `/led-status`, `/tinyml-status`) định kỳ từ 3 đến 5 giây, giúp cập nhật trạng thái thiết bị và dự đoán từ mô hình TinyML mà không làm gián đoạn trải nghiệm người dùng.

3.5.5 Quản lý tác vụ và Luồng xử lý (Task Management)

Tiến trình cốt lõi của máy chủ được thực thi bên trong hàm `main_server_task`, vận hành dựa trên các nguyên tắc của hệ điều hành thời gian thực FreeRTOS:

- **Vòng lặp điều phối (Main Loop):** Task duy trì một vòng lặp vô hạn để xử lý đồng thời các yêu cầu phân giải tên miền (`dnsServer.processNextRequest()`) và các yêu cầu từ máy khách HTTP (`server.handleClient()`).
  - **Cơ chế Non-blocking:** Logic kết nối Wi-Fi được thiết kế không chặn (non-blocking) bằng cách sử dụng hàm `millis()` để kiểm soát thời gian chờ (timeout 10 giây), giúp Web Server vẫn có thể đáp ứng các yêu cầu khác trong khi đang đợi thiết lập kết nối mạng.
  - **Tương tác phím cứng:** Hệ thống liên tục giám sát trạng thái của `BOOT_PIN`. Khi người dùng nhấn giữ nút này, thiết bị sẽ cưỡng bức chuyển về chế độ AP, giúp người dùng luôn có khả năng truy cập vào trang cấu hình kể cả khi mạng cục bộ gặp sự cố.
- **Giao tiếp liên luồng (Inter-task Communication):** Sau khi kết nối Wi-Fi thành công, hệ thống sử dụng cơ chế Semaphore (`xSemaphoreGive`) để gửi tín hiệu cho các Task khác biết rằng kết nối Internet đã sẵn sàng, cho phép kích hoạt các tính năng nâng cao như đồng bộ hóa dữ liệu lên Cloud.

3.6 Nhiệm vụ 5: Triển khai TinyML và đánh giá độ chính xác

3.6.1 Thu thập dữ liệu từ cảm biến (Data Collection)

Giai đoạn đầu tiên của hệ thống là thu thập dữ liệu thô từ cảm biến nhiệt độ và độ ẩm thông qua giao tiếp Serial. Kịch bản `collect_from_log.py` đóng vai trò là trình lắng nghe và xử lý dữ liệu từ vi điều khiển ESP32.

- **Giao tiếp Serial:** Sử dụng thư viện `pyserial` để thiết lập kết nối UART với Baudrate mặc định là 115200. Hệ thống liên tục đọc các dòng log từ cổng chỉ định (ví dụ: COM4).
- **Cơ chế phân tích (Parsing):** Để đảm bảo tính linh hoạt, kịch bản hỗ trợ ba định dạng đầu vào khác nhau thông qua hàm `parse_temp_humi_from_line`:
  1. **JSON:** Phân tích chuỗi có dạng `{"temperature": X, "humidity": Y}`.
  2. **Regex:** Sử dụng biểu thức chính quy `T\s*=\s*(...)\D+H\s*=\s*(...)` để tìm các mẫu như "T=30.5 H=65.2".
  3. **CSV Fallback:** Nếu không khớp các định dạng trên, hệ thống sẽ tự động tách chuỗi bằng dấu phẩy và lấy hai giá trị cuối cùng.
- **Lưu trữ dữ liệu:** Dữ liệu sau khi trích xuất thành công được gắn nhãn thời gian thực (ISO 8601) và ghi nối tiếp vào tệp `raw_data.csv` bằng `csv.DictWriter`.

Việc sử dụng `f.flush()` sau mỗi dòng ghi giúp đảm bảo dữ liệu không bị mất mát trong trường hợp kết nối Serial bị ngắt đột ngột.

3.6.2 Tiền xử lý và Gán nhãn dữ liệu (Data Preprocessing and Labeling)

Sau khi thu thập được dữ liệu thô, bước tiếp theo là làm sạch và gán nhãn tự động để tạo ra tập dữ liệu sẵn sàng cho việc huấn luyện mô hình (train-ready). Quá trình này được thực hiện thông qua kịch bản `extract_ready_data.py`.

- **Mục tiêu:** Chuyển đổi tệp `raw_data.csv` thành `train_ready.csv` bằng cách áp dụng các quy tắc logic nghiệp vụ để xác định trạng thái của hệ thống dựa trên thông số môi trường.
- **Quy tắc gán nhãn (Heuristic Labeling):** Hệ thống sử dụng hàm `classify_label` để phân loại dữ liệu vào các nhãn mục tiêu:
  - **Bật máy sưởi:** Khi nhiệt độ thấp hơn 24.0°C.
  - **Bật điều hòa:** Khi nhiệt độ lớn hơn 33.0°C và độ ẩm nằm trong khoảng 50% - 80%.
  - **Bật máy hút ẩm:** Khi nhiệt độ lớn hơn 33.0°C và độ ẩm vượt quá 80%.

- **Không hợp lệ:** Dữ liệu nằm ngoài dải vật lý (0 - 100) hoặc thiếu giá trị.
- **Chế độ bình thường:** Các trường hợp còn lại trong dải đo cho phép.

Kết quả của giai đoạn này là một tệp CSV gồm ba cột chính: `temperature`, `humidity`, và `label`, phục vụ trực tiếp cho các thuật toán học máy ở bước sau.

3.7 Đánh giá và So sánh Mô hình (Model Benchmarking)

Để tìm ra kiến trúc học máy phù hợp nhất cho phần cứng hạn chế tài nguyên, hệ thống tiến hành đánh giá toàn diện nhiều thuật toán khác nhau thông qua kịch bản `evaluate_models_comprehensive.py`. Quá trình này không chỉ xét đến độ chính xác mà còn đặc biệt chú trọng vào tài nguyên hệ thống.

- **Các mô hình thử nghiệm:** Kịch bản tiến hành huấn luyện và đối chiếu bốn mô hình đại diện:
  1. Gaussian Naive Bayes (GaussianNB).
  2. Random Forest.
  3. Support Vector Machine (SVM) với nhân RBF.
  4. Multi-Layer Perceptron (MLP) - Kích thước được ước lượng bằng cách lượng tử hóa trọng số về INT8 theo chuẩn TensorFlow Lite.
- **Chỉ số đánh giá ML tiêu chuẩn:** Các mô hình được đo lường hiệu suất thông qua Độ chính xác (Accuracy), Precision, Recall, F1-Score (trung bình có trọng số) và diện tích dưới đường cong ROC (ROC-AUC). Hệ thống cũng sử dụng K-Fold Cross-validation (với $K = 5$) để đánh giá tính ổn định của mô hình.
- **Chỉ số phần cứng (ESP32 Score):** Điểm đột phá của luồng công việc này là việc định nghĩa một công thức tính điểm (thang 100) dành riêng cho TinyML, bao gồm ba trọng số chính:
  - **Độ chính xác (40%):** Ưu tiên khả năng nhận diện đúng trạng thái.
  - **Kích thước mô hình (30%):** Dung lượng tính bằng KB. Mô hình $<$ 100KB được coi là lý tưởng; mô hình $>$ 1000KB bị trừ điểm mạnh do giới hạn bộ nhớ Flash/RAM của vi điều khiển.
  - **Tốc độ suy luận (30%):** Đánh giá thời gian dự đoán (ms) để đảm bảo đáp ứng thời gian thực.

Trực quan hóa và Báo cáo

Kết thúc quá trình đánh giá, hệ thống kết xuất một bảng xếp hạng (Ranking) và báo cáo JSON chi tiết. Nếu môi trường hỗ trợ `matplotlib`, các biểu đồ so sánh trực quan (như *Model Size Comparison* và *ESP32 Score*) sẽ tự động được tạo ra nhằm hỗ trợ lập trình viên ra quyết định lựa chọn thuật toán triển khai cuối cùng.

3.7.1 Huấn luyện và Xuất mã C (Model Training and C Header Export)

Dựa trên kết quả đánh giá ở giai đoạn trước, thuật toán Gaussian Naive Bayes (GaussianNB) được lựa chọn nhờ sự cân bằng tối ưu giữa độ chính xác và yêu cầu tài nguyên siêu nhỏ. Thay vì sử dụng các framework biên dịch nặng nề như TensorFlow Lite (TFLite) cho vi điều khiển, hệ thống thực thi một phương pháp triển khai trực tiếp thông qua kịch bản `export_naive_bayes.py`.

- **Triết lý triển khai (White-box TinyML):** GaussianNB không phải là một mạng nơ-ron tiêu chuẩn, do đó việc xuất sang định dạng TFLite không phải là giải pháp phù hợp. Thay vào đó, mô hình được "đóng băng"bằng cách trích xuất các tham số toán học đã học thành các mảng hằng số ngôn ngữ C (C arrays) và tự hiện thực hàm suy luận (score function) trực tiếp trên firmware của ESP32.
- **Chuẩn hóa Z-score:** Trước khi huấn luyện, dữ liệu đầu vào được chuẩn hóa sử dụng phương pháp Z-score. Các tham số trung bình (`mean`) và độ lệch chuẩn (`std`) được lưu lại để hệ thống nhúng có thể tiền xử lý dữ liệu cảm biến thực tế theo đúng tỷ lệ toán học.
- **Trích xuất ma trận cốt lõi:** Mô hình được huấn luyện bằng thư viện `scikit-learn`, sau đó trích xuất các thông số phân phối chuẩn bao gồm:
  - Xác suất tiên nghiệm của từng lớp (`class_prior_`).
  - Kỳ vọng (`theta_`) và Phương sai (`var_`) của từng đặc trưng (Nhiệt độ, Độ ẩm) tương ứng với từng trạng thái hệ thống.

Cấu trúc tệp tiêu đề C (C Header Output)

Kết quả của quá trình xuất là tệp `naive_bayes_model.h`, chứa toàn bộ tri thức của AI dưới dạng các mảng bộ nhớ tĩnh tối ưu cho SRAM của ESP32.

```
#ifndef NAIVE_BAYES_MODEL_H
#define NAIVE_BAYES_MODEL_H

#define NB_NUM_CLASSES 5
#define NB_NUM_FEATURES 2
```

```
static const float kNbFeatureMean[NB_NUM_FEATURES] = {47.44557571f, 50.07275391f};
static const float kNbFeatureStd[NB_NUM_FEATURES] = {30.14743614f, 28.64877701f};
// ... (Chi tiet cac mang kNbClassPrior, kNbTheta, kNbVariance) ...

// Runtime formula on ESP32:
// 1) normalize input: x_norm[j] = (x[j] - kNbFeatureMean[j]) / kNbFeatureStd[j]
// 2) score[class] = log(prior) - 0.5 * sum(log(2*pi*var)) - 0.5 *
   sum((x_norm-theta)^2 / var)
// 3) choose argmax(score)
#endif
```

Điểm nhấn kỹ thuật nằm ở phần chú thích cuối tệp. Nó cung cấp chính xác công thức toán học tính toán hàm Log-Likelihood, giúp lập trình viên nhúng dễ dàng dịch ngược (reverse-engineer) thuật toán ML thành mã C/C++ thuần túy mà không cần phải gọi đến bất kỳ thư viện hỗ trợ học máy nào bên ngoài.

3.7.2 Kiểm thử mô hình cục bộ (Local Model Testing)

Bước cuối cùng trước khi triển khai firmware lên phần cứng thực tế là kiểm chứng lại logic suy luận. Việc gỡ lỗi (debug) số học trực tiếp trên vi điều khiển thường tốn thời gian và khó khăn, do đó, kịch bản `test_naive_bayes.py` được thiết kế để giả lập lại chính xác môi trường tính toán của ESP32 ngay trên máy tính cá nhân.

- **Đồng bộ tham số (Parameter Synchronization):** Kịch bản không tải lại mô hình từ thư viện `scikit-learn`. Thay vào đó, nó đọc trực tiếp tệp `naive_bayes_model_metrics.json` (được sinh ra ở phần trước) chứa các ma trận như `zscore_mean`, `zscore_std`, `theta`, và `variance`. Điều này đảm bảo Python và C++ sử dụng chung một "bộ não"tham số.

- **Tái tạo logic toán học:** Kịch bản thực hiện lại các bước tính toán giống hệt mã C sẽ chạy trên ESP32: chuẩn hóa Z-score đầu vào và tính Log-Likelihood dựa trên phân phối chuẩn Gaussian.

- **Đánh giá độ tin cậy (Confidence Scoring):** Trong khi vi điều khiển có thể chỉ cần hàm `argmax()` để tìm nhãn có điểm cao nhất nhằm tiết kiệm chu kỳ CPU, kịch bản PC bổ sung thêm hàm `softmax` để chuyển đổi các giá trị log-likelihood thô thành xác suất phần trăm (%). Điều này giúp lập trình viên đánh giá độ "tự tin"của mô hình đối với từng dự đoán.

- **Giao diện tương tác (Interactive CLI):** Chương trình cung cấp giao diện dòng lệnh cho phép người dùng nhập tay các giá trị Nhiệt độ và Độ ẩm bất kỳ, sau đó in ra dự đoán tốt nhất (Top 1) kèm theo danh sách xếp hạng các khả năng khác (Top K).

Mã nguồn cốt lõi: Tính toán Log-Likelihood

Đoạn mã sau mô phỏng cách tính toán xác suất tiên nghiệm và hàm mật độ phân phối mà không dùng đến thư viện ML:

```
# Chuan hoa Z-score
x_norm = (x - mean) / np.where(std < 1e-6, 1.0, std)

# Tinh Gaussian Naive Bayes log likelihood
var = variance + epsilon
log_prior = np.log(np.clip(class_prior, 1e-12, None))
log_det = -0.5 * np.sum(np.log(2.0 * math.pi * var), axis=1)
quad = -0.5 * np.sum(((x_norm - theta) ** 2) / var, axis=1)

# Tong hop diem
logits = log_prior + log_det + quad
probs = softmax(logits) # Chuyen ve xac suat %
```

Nhờ cơ chế kiểm thử đối chứng này, lập trình viên có thể hoàn toàn tự tin biên dịch mã nguồn `C/C++` cho ESP32, đảm bảo rằng vi điều khiển sẽ đưa ra các quyết định phần cứng (như bật/tắt rơ-le điều hòa) chính xác như những gì thuật toán đã dự đoán trên máy tính.

3.7.3 Triển khai Firmware trên Vi điều khiển (On-device Deployment)

Sau khi mô hình được xuất sang mảng C và kiểm chứng logic thành công, bước cuối cùng là tích hợp "bộ não"này vào hệ điều hành thời gian thực (FreeRTOS) trên ESP32. Tệp mã nguồn `tinyml.cpp` đảm nhiệm vai trò là cầu nối giữa dữ liệu cảm biến thời gian thực và thuật toán Naive Bayes.

- **Kiến trúc đa tác vụ (Multitasking với FreeRTOS):** Thay vì chạy tuần tự trong vòng lặp `loop()` chặn (blocking) các tiến trình khác, hệ thống gói gọn quá trình suy luận vào một tác vụ độc lập mang tên `tiny_ml_task`.
  - Tác vụ này sử dụng cơ chế **Semaphore** (`xBinarySemaphoreTinyMLData`) để đồng bộ hóa. Nó sẽ ở trạng thái "ngủ"(Blocked) và không tiêu tốn chu kỳ CPU cho đến khi tác vụ đọc cảm biến (`temp_humi_monitor`) báo hiệu có dữ liệu mới.
- **Động cơ suy luận (Inference Engine):** Hàm `runNaiveBayes` hiện thực hóa lại chính xác các phương trình toán học đã đề cập ở Phần 4 và Phần 5 bằng thư viện `<math.h>` của ngôn ngữ C:
  1. **Chuẩn hóa:** Dữ liệu từ cảm biến thực (`glob_temperature`, `glob_humidity`) được chuẩn hóa Z-score qua hàm `normalizeInput` sử dụng hằng số `kNbFeatureMean` và `kNbFeatureStd`.

2. **Tính toán Log-Likelihood:** Sử dụng vòng lặp để cộng dồn xác suất theo công thức phân phối Gaussian.
3. **Hàm Softmax & Argmax:** Tính toán xác suất (%) cho từng lớp (`class_probs`) và chọn ra nhãn có xác suất cao nhất (`argmax`) làm kết quả dự đoán.

- **Cơ chế đánh giá On-device (On-the-fly Evaluation):** Một điểm sáng trong thiết kế phần mềm này là khối lệnh `tryUpdateGroundTruthFromSerial`. Nó biến ESP32 thành một công cụ tự đánh giá ngoài hiện trường. Thông qua cổng Serial, lập trình viên có thể gõ các lệnh như `gt 1` (gán nhãn đối chứng thực tế là 1) để ESP32 tự động đối chiếu dự đoán của nó với thực tế, từ đó thống kê số lượng dự đoán đúng/tổng số (`eval_correct` / `eval_total`).

Mã nguồn cốt lõi: Vòng lặp suy luận thời gian thực

Đoạn lệnh dưới đây minh họa cách mô hình ML hoạt động trơn tru trong môi trường RTOS mà không làm gián đoạn các luồng nghiệp vụ khác của hệ thống nhúng:

```
void tiny_ml_task(void *pvParameters) {
    setupTinyML();
    while (1) {
        // Kiem tra cong Serial de cap nhat nhan doi chung (neu co)
        tryUpdateGroundTruthFromSerial();

        // Cho doi tin hieu (Semaphore) tu cam bien
        if (xSemaphoreTake(xBinarySemaphoreTinyMLData, pdMS_TO_TICKS(10000)) !=
            pdTRUE) {
            continue;
        }

        // Chay suy luan AI
        runNaiveBayes(glob_temperature, glob_humidity);
        const int predicted_label = argmax(class_probs, NB_NUM_CLASSES);

        // Cap nhat bien toan cuc va in ket qua
        tinyml_predicted_label = predicted_label;
        Serial.print("T="); Serial.print(glob_temperature, 2);
        Serial.print(" H="); Serial.print(glob_humidity, 2);
        Serial.print(" | Label="); Serial.println(kNbLabels[predicted_label]);

        // Thong ke do chinh xac tai cho (neu bat Ground Truth)
        if (has_ground_truth) {
            ++eval_total;
```

```
            if (predicted_label == ground_truth_label) ++eval_correct;
        }

        vTaskDelay(kInferencePeriodTicks); // Chu ky suy luan (5 giay)
    }
}
```

Triết lý "White-box TinyML"đã được thể hiện trọn vẹn: Toàn bộ quá trình từ thu thập, huấn luyện, xuất thông số cho đến tính toán trên vi điều khiển hoàn toàn minh bạch, không phụ thuộc vào hộp đen (black-box API) nào, tối ưu hóa đến từng byte RAM và chu kỳ xung nhịp của vi xử lý.

3.8 Nhiệm vụ 6: Gửi dữ liệu lên máy chủ đám mây CoreIoT

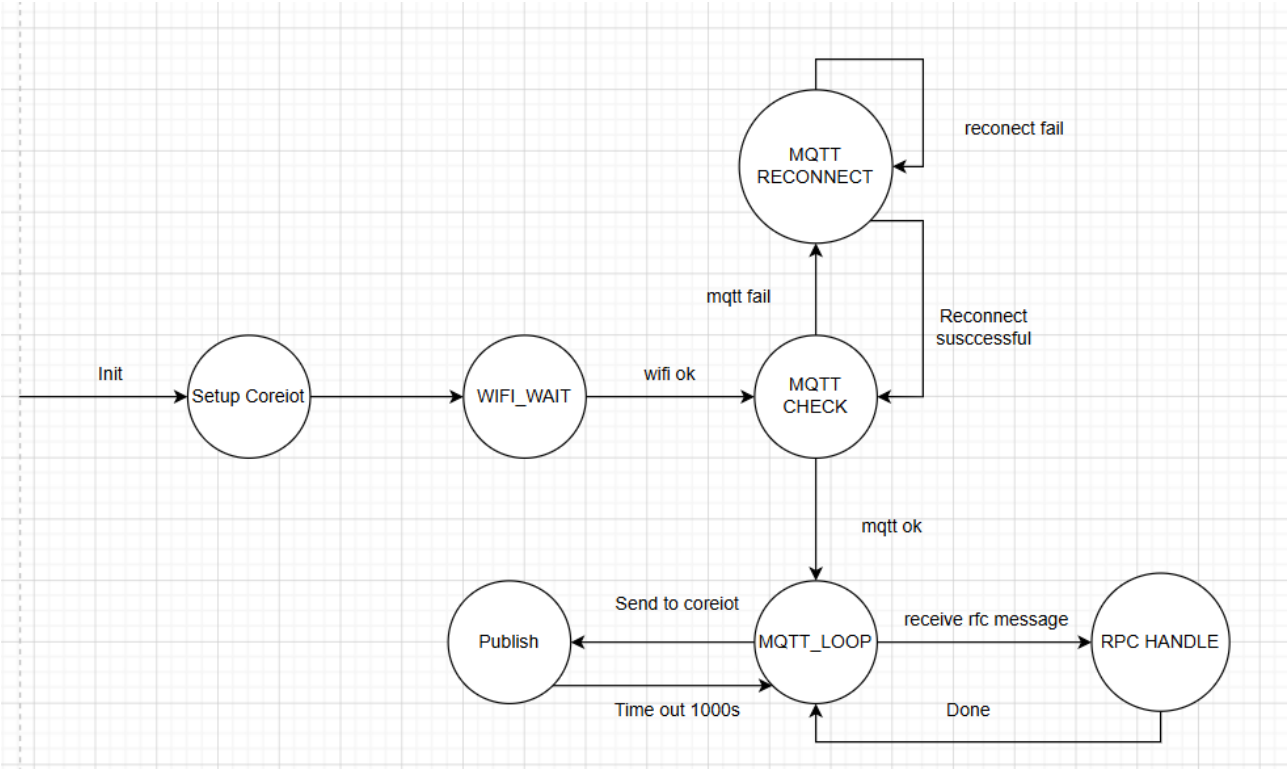
[Mô tả định dạng dữ liệu gửi lên đám mây, quy trình publish và chiến lược đảm bảo độ tin cậy/thử lại.]

3.8.1 Nguyên lý hoạt động

Tác vụ "CoreIoT"đóng vai trò là cầu nối giao tiếp hai chiều (Bidirectional Communication) giữa biên thiết bị (ESP32) và nền tảng đám mây (Cloud/ThingsBoard) thông qua giao thức MQTT ở port 1883. Tác vụ này thực hiện hai nhiệm vụ cốt lõi: báo cáo dữ liệu định kỳ và lắng nghe, thực thi các lệnh gọi hàm từ xa (RPC - Remote Procedure Call) từ người dùng.

- **Quản lý kết nối không xung đột:** Để tránh hiện tượng "tranh chấp"module Wi-Fi với Web Server gây treo hệ thống, Task CoreIoT không tự gọi hàm `WiFi.begin()`. Thay vào đó, nó liên tục giám sát cờ `isWifiConnected` do `main_server_task` cấp phát. Nếu mất Wi-Fi, tác vụ tự động chuyển sang trạng thái chờ 2000ms; chỉ khi có mạng, quá trình kết nối MQTT (`reconnect`) mới được phép kích hoạt.
- **Đồng bộ:** Khi duy trì kết nối MQTT ổn định, cứ mỗi 10.000ms (10 giây), hệ thống sẽ đóng gói các thông số môi trường hiện hành (`glob_temperature`, `glob_humidity`) và cấu hình ngưỡng cảnh báo (`glob_temp_threshold`) thành chuỗi JSON. Chuỗi này được gửi lên topic `v1/devices/me/telemetry` để hiển thị lên Dashboard của hệ thống CoreIoT.
- **Thực thi lệnh điều khiển RPC (RPC Callback):** Hệ thống đăng ký (subscribe) lắng nghe tại topic `v1/devices/me/rpc/request/+`. Khi có lệnh điều khiển từ Dashboard (VD: kéo Slider thay đổi ngưỡng), hàm `callback` sẽ được ngắt để gọi. Thư viện `ArduinoJson` phân tích gói tin, nhận diện phương thức `"setThre"` và ngay lập tức trích xuất tham số `params` để cập nhật đè lên biến `glob_temp_threshold`. Nhờ đó, thay đổi từ Cloud lập tức được phản ánh xuống giới hạn cảnh báo của màn hình LCD ở biên thiết bị.

3.8.2 Sơ đồ máy trạng thái (FSM)



Hình 4: Task 6 FSM

3.9 Tổng quan kiến trúc

[Trình bày kiến trúc tổng thể của hệ thống, luồng dữ liệu và sự tương tác giữa các tác vụ.]

3.10 Tính mới và cải tiến

[Nêu bật các tính năng sáng tạo, nâng cấp giao diện và các mở rộng TinyML để tăng điểm cộng.]

3.11 Tóm tắt chương

[Tóm tắt nội dung chương triển khai.] Chương này tổng kết quá trình triển khai hệ thống trên ESP32. Thông qua FreeRTOS, hệ thống đã vận hành đa nhiệm đồng bộ các module phần cứng: cảm biến DHT20, màn hình LCD và hệ thống LED cảnh báo trực quan. Nền tảng mạng được thiết lập linh hoạt, kết hợp Máy chủ Web cục bộ để cấu hình thiết bị và giao tiếp đám mây CoreIoT qua giao thức MQTT. Đặc biệt, việc nhúng thành công mô hình học máy TinyML tại biên đã nâng cấp thiết bị thành một hệ thống AIoT thông minh, tự chủ suy diễn dữ liệu môi trường theo thời gian thực.

4 Đánh giá và kết quả thực nghiệm

4.1 Giới thiệu

[Giới thiệu chương đánh giá và các thước đo được sử dụng.]

4.2 Các kịch bản kiểm thử

[Mô tả các kịch bản kiểm thử: mạng yếu, mạng không ổn định và các điều kiện cảm biến thay đổi.]

4.3 Kết quả chức năng

[Trình bày tính đúng đắn và giải thích các hành vi chức năng quan sát được.]

4.4 Đánh giá TinyML

[Đánh giá độ chính xác, độ trễ và thời gian phản hồi của pipeline TinyML.]

4.5 Thảo luận

[Thảo luận về điểm mạnh, điểm nghẽn và các đánh đổi quan sát được trong thực nghiệm.]

4.6 Tóm tắt chương

[Tóm tắt nội dung chương đánh giá.]

5 Kết luận và thảo luận nhóm

5.1 Thảo luận và kết luận

[Tổng kết quá trình triển khai, các thách thức, bài học kinh nghiệm và kết luận cuối cùng.]

5.2 Hướng phát triển trong tương lai

[Đề xuất các cải tiến trong tương lai hoặc một hướng triển khai lại, ví dụ MicroPython.]

Tài liệu tham khảo

Tài liệu

[1] [Bổ sung tài liệu tham khảo 1]

[2] [Bổ sung tài liệu tham khảo 2]

A Phụ lục A: Tổ chức nhóm

[Bổ sung vai trò từng thành viên, trách nhiệm và thông tin đóng góp Git tại đây.]

B Phụ lục B: Tài liệu bổ sung

[Bổ sung sơ đồ khối, trích đoạn mã nguồn và các kết quả phụ trợ tại đây.]